

Lab Tutorial 5.2 – Named Pipes

Ordinary pipe in Linux and Unnamed pipe in Windows are communication mechanisms between parent and child processes. In broader problems, such as communication between processes that do not have a parent-child relationship, named pipes can be used.

Target	Related theory	Resources
Communicate in the shell	Ch3: Process Ch3.3.7.4 Pipes	https://github.com/Trantin84/LAB_IntroOS (example source code).
Named pipes		Use Ubuntu 16/18 image
Simple chat program		Use Windows 10

Ask students: Understand the theory of inter-process communication using named pipes. Understand and execute the provided code snippets. Compare normal/unnamed pipes with named pipes. Analyze and apply to inter-process communication problems.

Student reviews: Questions and answers about theoretical issues. Practical skills. Exercise.

Submission requirements: .c source code files and .out executable files of the Practice Session Requirements and Final Tutorial Exercises within the time allowed by the instructor.

Preferences

[1] Abraham Silberschatz, Peter B. Galvin, Greg Gagne, [2018], Operating System Concepts, 10th edition, John Wiley & Sons, New Jersey.

Programming Problems of Chapter 3.

[2] StackOverflow, [2021], Example of using named pipes in Linux Bash,

Access <https://stackoverflow.com/questions/4113986/example-of-using-named-pipes-in-linux-bash>

[3] Geeksforgeeks, [2021], Named Pipe or FIFO with example C program,

Access <https://www.geeksforgeeks.org/named-pipe-fifo-example-c-program/>

Communicate through CLI.

Requirements 1: Create a named pipe and communicate between two processes through that pipe.

Instruct: Open 2 separate Terminals.

Type the command on Terminal 1

```
| >mkfifo mypipe  
| >echo Hello TDTU > mypipe
```

Switch to Terminal 2, type the command

```
| >cat mypipe
```

The prompt will print "Hello TDTU". You can continue to communicate by sending in "String > mypipe" and reading out as "cat mypipe". After finishing communication, destroy the pipe with the command:

```
| >unlink mypipe
```

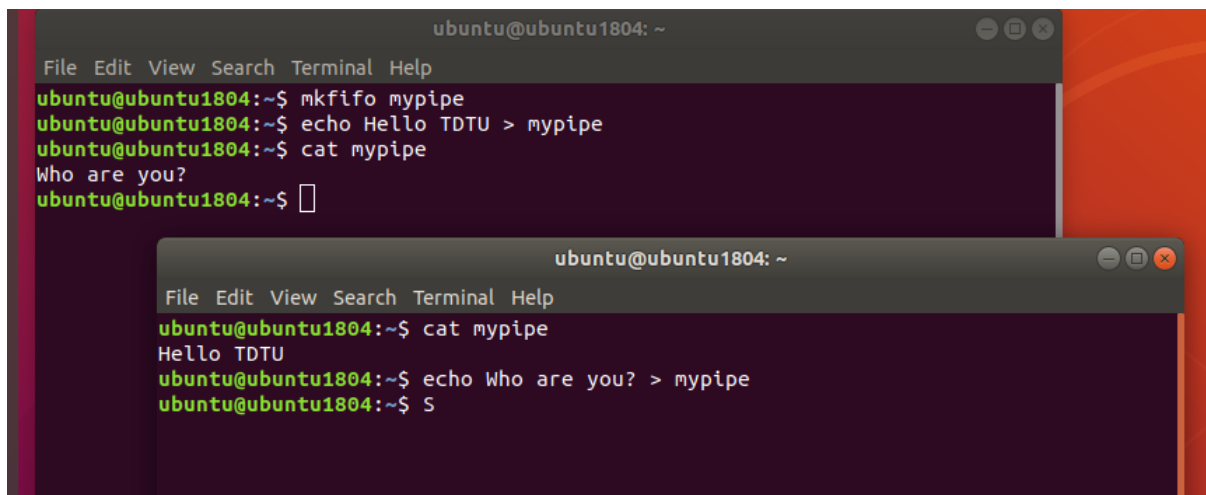


Image1. Communication between two independent bash processes.

Ordinary pipe

Requirement 2: Write a chat program between two independent processes.

Instruct: Download the files LAB_IntroOS/LAB_5/task2_2parent.c and /task2_2child.c, compile and execute on 2 different terminals, the parent process executes first.

a. PARENT source code explanation

- Lines 22, 23 and 24 define 2 pipes and full access (0666)
- Line 26 declares a maximum pipe size of 4096 bytes.
- Line 29 declares the strings s1, s2, buf with sizes equal to the pipe size.
- Line 30 contains resistors that manage the read and write heads of the created pipeline.
- Line 31 is the call to create a pipe with path FIFO1, read and write permission PM.
- Line 36 is the opening write end (side 1) of the FIFO1 pipeline.
- Line 38 opens the reader (side 0) of the FIFO2 pipeline.
- Line 44 records the string s1 and the pipe created in line 36.
- Line 45 is to read all (PIPE_BUF is 4096 bytes) of the pipe and write to s2.
- Lines 50 and 51 close the read and write heads of the pipeline.
- Lines 52 and 54 are to unlink the pipeline, after closing both read and write heads of each pipeline.

22	#define FIFO1 "/tmp/ff.1"
23	#define FIFO2 "/tmp/ff.2"
24	#define PM 0666
26	#define PIPE_BUF 4096
29	char s1[PIPE_BUF], s2[PIPE_BUF], buf[PIPE_BUF];
30	int readfd, writefd;
31	if ((mknod(FIFO1, S_IFIFO PM, 0) < 0) && (errno != EEXIST))
36	if ((writefd = open(FIFO1, 1)) < 0)
38	if ((readfd = open(FIFO2, 0)) < 0)
44	write(writefd, s1, strlen(s1));
45	read(readfd, s2, PIPE_BUF);
50	close(readfd);
51	close(writefd);
52	if (unlink(FIFO1) < 0)
54	if (unlink(FIFO2) < 0)

b. CHILD source code explanation

- Line 31 is to create a FIFO2 pipe to send data in the opposite direction.
- The remaining lines perform the same work as the Parent process, pay attention to which pipe the operation takes place on.

31	if ((mknod(FIFO2, S_IFIFO PM, 0) < 0) && (errno != EEXIST)) {
----	---

```

36     if ((readfd = open(FIFO1, 0)) < 0)
37     if ((writefd = open(FIFO2, 1)) < 0)
41     read(readfd, s2, PIPE_BUF);
46     write(writefd, s1, strlen(s1));
50     close(readfd);
51     close(writefd);

```

```

File Edit View Search Terminal Help
ubuntu@ubuntu1804:~/tttin/LAB5$ ./parent.out
(me): Hello student.
Child said: Hi teacher, how are you today?
(me): Thanks. I'm fine. Have you finished your homework?
Child said: Not yet. I feel sick. Might I've got COVID-19.
(me): Oh no no. Stay at home and call 18001119.
Child said: Ok teacher.
eel sick. Might I've got COVID-19.
(me): byebye
ubuntu@ubuntu1804:~/tttin/LAB5$

ubuntu@ubuntu1804: ~/tttin/LAB5
File Edit View Search Terminal Help
ubuntu@ubuntu1804:~/tttin/LAB5$ ./child.out
Parent said: Hello student.
(me): Hi teacher, how are you today?
Parent said: Thanks. I'm fine. Have you finished your homework?
(me): Not yet. I feel sick. Might I've got COVID-19.
Parent said: Oh no no. Stay at home and call 18001119.
omework?
(me): Ok teacher.
byebye
Parent said: Oh no no. Stay at home and call 18001119.
omework?
(me): ubuntu@ubuntu1804:~/tttin/LAB5$

```

Image2. Chat program using Named Pipes.

Programming exercises.

1. The parent process passes the first argument (argv[1]) which is an integer greater than 3 to the child process through the named pipe. The child process receives and calculates the value $n! = 1 * 2 * \dots * n$ and write it to the pipe. The parent process receives and outputs data to the screen. Use a normal pipe (Named Pipe).

```

> ./baitap2A.out 4
4! = 24

```